AI Applications Lecture 18

Parameter Efficient Fine Tuning (PEFT) and LoRA

SUZUKI, Atsushi[a]

2026-11-12

[a]I thank my collaborator, Jing WANG, for cooperating in creating the teaching materials.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Background

Learning Outcomes

At the end of this lecture, you will be able to

- Explain the objective of parameter-efficient fine-tuning.
- Modify the behavior of a NN using low-rank adaptation (LoRA).
- Apply LoRA to a practical generative AI.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Preliminaries: mathematical notations

Set:

- Sets: $\mathcal{A}$
- Membership: $x \in \mathcal{A}$
- Empty set: $\{\}$
- Roster notation: $\{a, b, c\}$
- Set-builder: $\{x \in \mathcal{A} | P(x)\}$
- Cardinality: $|\mathcal{A}|$
- Real numbers: $\mathbb{R}, \mathbb{R}_{>0}, \mathbb{R}_{\geq 0}$
- Integers: $\mathbb{Z}, \mathbb{Z}_{>0}, \mathbb{Z}_{\geq 0}$
- Integer range: $\{1, 2, ..., k\}$

Function:

- $f : \mathcal{X} \to \mathcal{Y}$: f maps from $\mathcal{X}$ to $\mathcal{Y}$.
- $y = f(x)$: The output of f for input x.

Definition:

- $(\mathrm{LHS}) \coloneqq (\mathrm{RHS})$: Left side is defined by the right side.

Preliminaries: mathematical notations

Sequence: Denoted by $\boldsymbol{a} = (a_0, a_1, \ldots, a_{n-1})$.

- A function $\boldsymbol{a} : \{0, 1, ..., n-1\} \to \mathcal{A}$.
- Length is denoted by $|\boldsymbol{a}|$.

Vector: Denoted by $\boldsymbol{v}$.

- A column of numbers, $\boldsymbol{v} \in \mathbb{R}^n$.
- i-th element is v_i.

Matrix: Denoted by $\boldsymbol{A}$.

- $m \times n$ matrix: $\boldsymbol{A} \in \mathbb{R}^{m,n}$.
- (i, j)-th element is $a_{i,j}$.
- Transpose: $\boldsymbol{A}^\top$.

Tensor: Denoted by $\underline{\boldsymbol{A}}$.

- Simply a multi-dimensional array.
- Vector $\to$ 1st-order, Matrix $\to$ 2nd-order.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Question: Why do we apply parameter-efficient fine tuning? How effective?

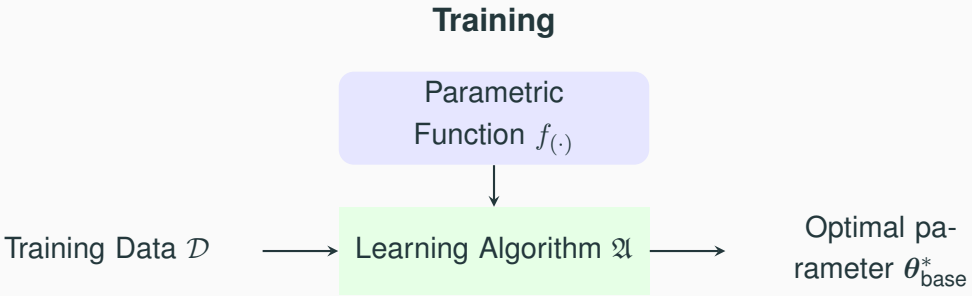
AI as a parametric function: review

Let $\mathcal{X}$ be the input set, $\mathcal{Y}$ the output set, and $\boldsymbol{\theta}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{pretrain}}}$ a parameter vector. A parametric model is a function $f_{\boldsymbol{\theta}_{\text{pretrain}}} : \mathcal{X} \longrightarrow \mathcal{Y}$, mapping $\boldsymbol{x} \longmapsto f_{\boldsymbol{\theta}_{\text{pretrain}}}(\boldsymbol{x})$. For supervised data $D = ((\boldsymbol{x}_i, y_i))_{i=1}^n$ and a loss $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\geq 0}$, full training solves [1]

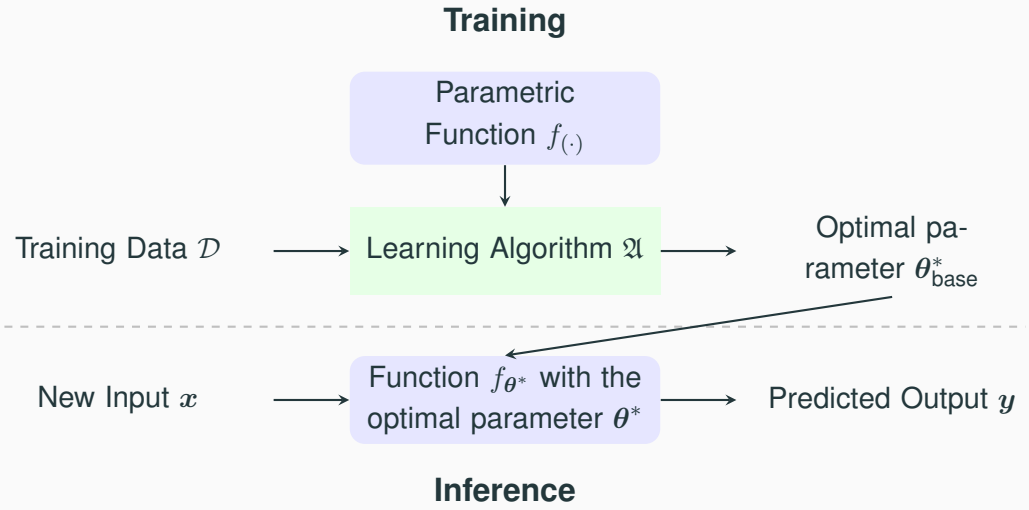
$$\boldsymbol{\theta}^*_{\text{full}} \in \underset{\boldsymbol{\theta}_{\text{full}} \in \mathbb{R}^{d_{\text{pretrain}}}}{\arg\min} \frac{1}{n} \sum_{i=1}^{n} \ell(f_{\boldsymbol{\theta}_{\text{full}}}(\boldsymbol{x}_i), y_i) \,.$$

- **Training**: determine $\boldsymbol{\theta}^*_{\text{full}}$ from D.
- **Inference**: for an unseen $\boldsymbol{x} \in \mathcal{X}$, compute $f_{\boldsymbol{\theta}^*_{\text{full}}}(\boldsymbol{x})$ without changing the parameters.

AI as a parametric function: review



AI as a parametric function: review



Parameter-efficient fine-tuning: definition

Start from pre-trained, fixed parameters $\boldsymbol{\theta}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{pretrain}}}$.

Introduce a low-dimensional parameter $\boldsymbol{\theta}_{\text{PEFT}} \in \mathbb{R}^{d_{\text{PEFT}}}$, where $d_{\text{PEFT}} \ll d_{\text{pretrain}}$ [2, 3].

From the original parametric function (NN) $f_{(\cdot)}$, PEFT first defines $f^{\text{PEFT}}_{(\cdot,\cdot)} : \mathcal{X} \to \mathcal{Y}$ taking two parameter vectors $\boldsymbol{\theta}_{\text{pretrain}}$ and $\boldsymbol{\theta}_{\text{PEFT}}$.

Inference: Suppose we get a good PEFT parameter $\boldsymbol{\theta}_{\text{PEFT}}$. For an input $\boldsymbol{x}$, we obtain an output by $f^{\text{PEFT}}_{\boldsymbol{\theta}_{\text{pretrain}}, \boldsymbol{\theta}_{\text{PEFT}}}(\boldsymbol{x})$.

Training: We fix $\boldsymbol{\theta}_{\text{pretrain}}$ and tries to find a good $\boldsymbol{\theta}_{\text{PEFT}}$ so $f^{\text{PEFT}}_{\boldsymbol{\theta}_{\text{pretrain}}, \boldsymbol{\theta}_{\text{PEFT}}}$ performs well.

Why use PEFT?

1. **Easy reversion.**

Why use PEFT?

1. **Easy reversion.** If we store θ_{pretrain} and $\boldsymbol{\theta}_{\text{PEFT}}$, we can use finetuned model $f^{\text{PEFT}}_{\boldsymbol{\theta}_{\text{pretrain}},\boldsymbol{\theta}_{\text{PEFT}}}$ and the original model $f_{\boldsymbol{\theta}_{\text{pretrain}}}$. We do NOT need to store two d_{pretrain}-dimensional vectors.
2. **Reducing required storage**.

Why use PEFT?

1. **Easy reversion.** If we store $\boldsymbol{\theta}_{\text{pretrain}}$ and $\boldsymbol{\theta}_{\text{PEFT}}$, we can use finetuned model $f^{\text{PEFT}}_{\boldsymbol{\theta}_{\text{pretrain}},\boldsymbol{\theta}_{\text{PEFT}}}$ and the original model $f_{\boldsymbol{\theta}_{\text{pretrain}}}$. We do NOT need to store two d_{pretrain}-dimensional vectors.
2. **Reducing required storage**. The pretrained d_{pretrain}-dimensional parameter vector can be shared between tasks. When we have K tasks:
   - Naive fine-tuning requires Kd_{pretrain} scalar parameters.
   - PEFT approach only requires $d_{\text{pretrain}} + Kd_{\text{PEFT}}$ scalar parameters.
3. **Easy training.**

Why use PEFT?

1. **Easy reversion.** If we store $\boldsymbol{\theta}_{\text{pretrain}}$ and $\boldsymbol{\theta}_{\text{PEFT}}$, we can use finetuned model $f^{\text{PEFT}}_{\boldsymbol{\theta}_{\text{pretrain}},\boldsymbol{\theta}_{\text{PEFT}}}$ and the original model $f_{\boldsymbol{\theta}_{\text{pretrain}}}$. We do NOT need to store two d_{pretrain}-dimensional vectors.
2. **Reducing required storage**. The pretrained d_{pretrain}-dimensional parameter vector can be shared between tasks. When we have K tasks:
   - Naive fine-tuning requires Kd_{pretrain} scalar parameters.
   - PEFT approach only requires $d_{\text{pretrain}} + Kd_{\text{PEFT}}$ scalar parameters.
3. **Easy training.** An optimization algorithm updates d_{PEFT} rather than d_{pretrain} scalar parameters. This can reduce optimization cost and increase optimization stability. Thus, training can be done on a small computer and small dataset.

Stable Diffusion 1.5: concrete adapter size

Example (SD 1.5-compatible model and Minimalist Anime Style LoRA)

Consider Stable Diffusion 1.5 as an example. Its (pretrained) parameter counts are $d_{\text{pretrain}} = 1{,}066{,}235{,}307 (\approx 1.07\text{B})$.

Take Minimalist Anime Style LoRA as an example: Its parameter counts are

Stable Diffusion 1.5: concrete adapter size

Example (SD 1.5-compatible model and Minimalist Anime Style LoRA)

Consider Stable Diffusion 1.5 as an example. Its (pretrained) parameter counts are $d_{\mathsf{pretrain}} = 1{,}066{,}235{,}307 (\approx 1.07\mathrm{B})$.

Take Minimalist Anime Style LoRA as an example: Its parameter counts are $d_{\mathsf{PEFT}} = 75{,}497{,}736 (\approx 75\mathrm{M})$. It's just

Stable Diffusion 1.5: concrete adapter size

Example (SD 1.5-compatible model and Minimalist Anime Style LoRA)

Consider Stable Diffusion 1.5 as an example. Its (pretrained) parameter counts are $d_{\text{pretrain}} = 1{,}066{,}235{,}307 (\approx 1.07\text{B})$.

Take Minimalist Anime Style LoRA as an example: Its parameter counts are $d_{\text{PEFT}} = 75{,}497{,}736 (\approx 75\text{M})$. It's just about 7.08% of the pretrained model!

Source: Minimalist Anime Style, `https://civitai.com/models/24833/minimalist-anime-style`; file `anime_minimalist_v1-000020.safetensors`, `https://huggingface.co/hyu8828/Minimalist-Anime/tree/main`; SafeTensors metadata and tensor-shape audit.

Exercise

A pretrained model has 1B parameters; each PEFT adapter has 75M. Using FP16 and 15GB of Google Drive storage, how many tasks can we store with (i) full fine-tuning and (ii) one shared base model plus adapters? Assume 1 GB $= 10^9$ bytes.

Answer: FP16 uses 2 bytes per parameter. Full fine-tuning: $\lfloor 15/(1 \cdot 2) \rfloor = 7$ tasks. PEFT: $\lfloor (15 - 2)/(0.075 \cdot 2) \rfloor = 86$ tasks.

PEFT Advantages — Answer

Question: Why do we apply parameter-efficient fine tuning? How effective?

Answer:

PEFT Advantages — Answer

Question: Why do we apply parameter-efficient fine tuning? How effective?

Answer: We can represent an AI behavior change with a few parameters. Specifically, it can

PEFT Advantages — Answer

Question: Why do we apply parameter-efficient fine tuning? How effective?

Answer: We can represent an AI behavior change with a few parameters. Specifically, it can

- easily revert the modification by “detaching” the parameter difference.

PEFT Advantages — Answer

Question: Why do we apply parameter-efficient fine tuning? How effective?

Answer: We can represent an AI behavior change with a few parameters. Specifically, it can

- easily revert the modification by “detaching” the parameter difference.
- reduce the data size to store the parameter difference from the original model.

PEFT Advantages — Answer

Question: Why do we apply parameter-efficient fine tuning? How effective?

Answer: We can represent an AI behavior change with a few parameters. Specifically, it can

- easily revert the modification by “detaching” the parameter difference.
- reduce the data size to store the parameter difference from the original model.
- be achieved by training on a small computer and small dataset.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Question: PEFT attempts to achieve finetuning only by small amount of parameters; it cannot change the whole AI model. Within this constraint, how

Question: PEFT attempts to achieve finetuning only by small amount of parameters; it cannot change the whole AI model. Within this constraint, how can we modify the AI behavior effectively?

PEFT's drawback

In a PEFT scheme, we may represent the finetuning by parameters of 7% of the original model.

PEFT's drawback

In a PEFT scheme, we may represent the finetuning by parameters of 7% of the original model.

There is no guarantee that we can change the AI's behavior as we want.

What

PEFT's drawback

In a PEFT scheme, we may represent the finetuning by parameters of 7% of the original model.

There is no guarantee that we can change the AI's behavior as we want.

What trick enables us to modify the AI effectively with a small number of parameters?

PEFT's drawback

In a PEFT scheme, we may represent the finetuning by parameters of 7% of the original model.

There is no guarantee that we can change the AI's behavior as we want.

What trick enables us to modify the AI effectively with a small number of parameters?

Representative technique: Low rank adaptation (LoRA)

LoRA: a low-rank adaptation

A modern NN architecture has many matrix parameters, e.g., scaled-dot attention, 1x1 convolution, MLP, etc.
The low-rank adaptation (LoRA) [2] replaces a matrix parameter $\boldsymbol{W}_{\text{pretrain}}$ in a NN by

$$\boldsymbol{W}_{\text{pretrain}} + \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}.$$

Here,

LoRA: a low-rank adaptation

A modern NN architecture has many matrix parameters, e.g., scaled-dot attention, 1x1 convolution, MLP, etc.
The low-rank adaptation (LoRA) [2] replaces a matrix parameter $\boldsymbol{W}_{\text{pretrain}}$ in a NN by

$$\boldsymbol{W}_{\text{pretrain}} + \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}.$$

Here, $\boldsymbol{A}$ and $\boldsymbol{B}$ consist of r vectors and r column vectors, respectively, where r is a small integer called the **rank**.

LoRA: a low-rank adaptation

A modern NN architecture has many matrix parameters, e.g., scaled-dot attention, 1x1 convolution, MLP, etc.
The low-rank adaptation (LoRA) [2] replaces a matrix parameter $\boldsymbol{W}_{\text{pretrain}}$ in a NN by

$$\boldsymbol{W}_{\text{pretrain}} + \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}.$$

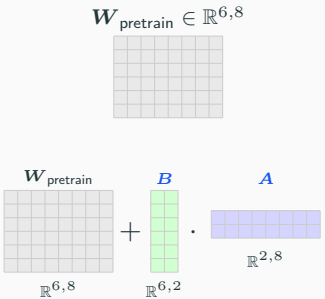
Here, $\boldsymbol{A}$ and $\boldsymbol{B}$ consist of r vectors and r column vectors, respectively, where r is a small integer called the **rank**.
Only $\boldsymbol{A}$ and $\boldsymbol{B}$ are trained in LoRA.

LoRA: a low-rank adaptation

A modern NN architecture has many matrix parameters, e.g., scaled-dot attention, 1x1 convolution, MLP, etc.
The low-rank adaptation (LoRA) [2] replaces a matrix parameter $\boldsymbol{W}_{\text{pretrain}}$ in a NN by

$$\boldsymbol{W}_{\text{pretrain}} + \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}.$$

Here, $\boldsymbol{A}$ and $\boldsymbol{B}$ consist of r vectors and r column vectors, respectively, where r is a small integer called the **rank**.
Only $\boldsymbol{A}$ and $\boldsymbol{B}$ are trained in LoRA.
η can control the modification and is determined at the inference phase.



Source: https://github.com/huggingface/peft/blob/main/src/peft/tuners/lora/layer.py#L2584-L2613.

What does LoRA save?

- General finetuning

What does LoRA save?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \boldsymbol{\Delta W}$, where $\boldsymbol{\Delta W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} := \frac{\eta}{2} \boldsymbol{B} \boldsymbol{A}$.

What does LoRA save?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

How many scalar parameters do we need to save for each?

- General finetuning: $d_{\text{out}}, d_{\text{in}}$ scalar parameters.
- LoRA: $d_{\text{out}} r + r d_{\text{in}} = r(d_{\text{out}} + d_{\text{in}})$ scalar parameters.

Exercise

For $d_{\text{out}} = 1000$, $d_{\text{in}} = 100$, and $r = 10$, compare the parameter counts of full fine-tuning and LoRA.

Answer: Full: $1000 \cdot 100 = 100{,}000$. LoRA: $10(1000 + 100) = 11{,}000$ (11% of full).

Exercise: which input vector does LoRA change?

Exercise

Let $d_{\text{in}} = 3$, $d_{\text{out}} = 4$, $r = 2$, and $\eta = 1$, $\alpha = r = 2$. Define

$$\boldsymbol{W}_{\text{pretrain}} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix} \in \mathbb{R}^{4,3}, \quad \boldsymbol{A} = \begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & -1 \end{pmatrix} \in \mathbb{R}^{2,3}, \quad \boldsymbol{B} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \\ 2 & 1 \end{pmatrix} \in \mathbb{R}^{4,2}.$$

For $\boldsymbol{x}^{(1)} = (1, 2, 1)^\top \in \mathbb{R}^3$ and $\boldsymbol{x}^{(2)} = (-1, 1, 1)^\top \in \mathbb{R}^3$,
compute $\boldsymbol{W}_{\text{pretrain}}\boldsymbol{x}$ and $(\boldsymbol{W}_{\text{pretrain}} + \boldsymbol{B}\boldsymbol{A})\boldsymbol{x}$.

Also, observe when LoRA changes the output.

Exercise: answer and verification

Answer:

$$\boldsymbol{W}_{\text{pretrain}}\boldsymbol{x}^{(1)} = (1,2,1,4)^\top, \quad (\boldsymbol{W}_{\text{pretrain}} + \boldsymbol{BA})\boldsymbol{x}^{(1)} = (6,3,5,15)^\top,$$
$$\boldsymbol{W}_{\text{pretrain}}\boldsymbol{x}^{(2)} = (-1,1,1,1)^\top, \quad (\boldsymbol{W}_{\text{pretrain}} + \boldsymbol{BA})\boldsymbol{x}^{(2)} = (-1,1,1,1)^\top.$$

Since $\boldsymbol{A}\boldsymbol{x}^{(2)} = \boldsymbol{0}$, LoRA does not change the output for $\boldsymbol{x}^{(2)}$.

What does LoRA sacrifices?

- General finetuning

What does LoRA sacrifices?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} := \frac{\eta}{2} \boldsymbol{B} \boldsymbol{A}$.

Clearly, LoRA sacrifices

What does LoRA sacrifices?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} := \frac{\eta}{2} \boldsymbol{B} \boldsymbol{A}$.

Clearly, LoRA sacrifices flexibility in representing the fine-tuning difference $\Delta \boldsymbol{W}$.

What does LoRA sacrifices?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

Clearly, LoRA sacrifices flexibility in representing the fine-tuning difference $\Delta\boldsymbol{W}$.

In what sense is LoRA's representation limited?

What does LoRA sacrifices?

- General finetuning replaces $\boldsymbol{W}_{\text{pretrain}} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} \in \mathbb{R}^{d_{\text{out}}, d_{\text{in}}}$ may take any form.
- LoRA uses $\boldsymbol{A} \in \mathbb{R}^{r, d_{\text{in}}}$ and $\boldsymbol{B} \in \mathbb{R}^{d_{\text{out}}, r}$ with small r. It replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta \boldsymbol{W}$, where $\Delta \boldsymbol{W} := \frac{\eta}{2} \boldsymbol{B} \boldsymbol{A}$.

Clearly, LoRA sacrifices flexibility in representing the fine-tuning difference $\Delta \boldsymbol{W}$.

In what sense is LoRA's representation limited?

Answer: LoRA can only change the behavior for an input in a low-dimensional subspace.

LoRA cannot change the behavior for all possible inputs

Suppose $\boldsymbol{A} \in \mathbb{R}^{r,d_{\text{in}}}, \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}},r},$.

LoRA cannot change the behavior for all possible inputs

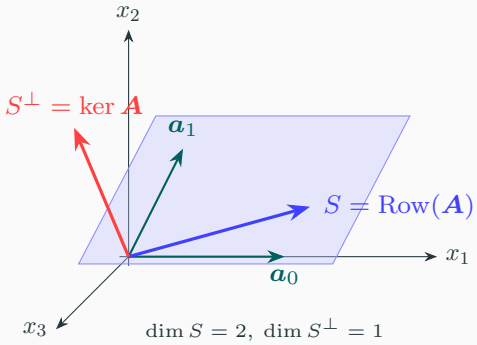
Suppose $\boldsymbol{A} \in \mathbb{R}^{r,d_{\text{in}}}, \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}},r},$.
What input can the LoRA change and what cannot?

LoRA cannot change the behavior for all possible inputs

Suppose $\boldsymbol{A} \in \mathbb{R}^{r,d_{\text{in}}}, \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}},r},$.
What input can the LoRA change and what cannot?
Let $S := \text{span}(\text{rows of } \boldsymbol{A})$.

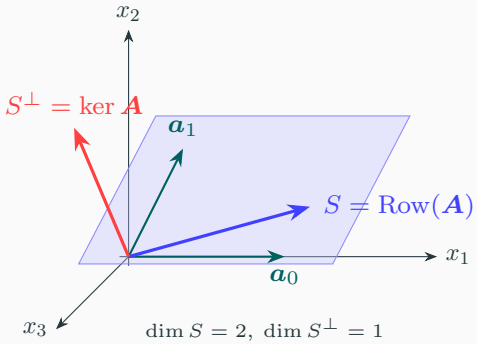
LoRA cannot change the behavior for all possible inputs

Suppose $\boldsymbol{A} \in \mathbb{R}^{r,d_{\text{in}}}, \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}},r},$.
What input can the LoRA change and what cannot?
Let $S := \text{span}(\text{rows of } \boldsymbol{A})$.
Define its orthogonal complement $S^\perp := \{\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}} \mid \boldsymbol{x}^\mathsf{T}\boldsymbol{a} \text{ for all } \boldsymbol{a} \in \text{rows of } \boldsymbol{A}\}$.



LoRA cannot change the behavior for all possible inputs

Suppose $\boldsymbol{A} \in \mathbb{R}^{r,d_{\text{in}}}, \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}},r},$.
What input can the LoRA change and what cannot?
Let $S := \text{span}(\text{rows of } \boldsymbol{A})$.
Define its orthogonal complement $S^\perp := \{\boldsymbol{x} \in \mathbb{R}^{d_{\text{in}}} \mid \boldsymbol{x}^\mathsf{T}\boldsymbol{a} \text{ for all } \boldsymbol{a} \in \text{rows of } \boldsymbol{A}\}$.



If $\boldsymbol{x} \in S^\perp$, then $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$. Thus, $\Delta \boldsymbol{W}\boldsymbol{x} = \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}$.

This means the LoRA cannot change the input in $S^\perp$.

Moreover, $\dim S \leq r$, so $\dim S^\perp \geq d_{\text{in}} - r$. LoRA sacrifices representing changes for inputs in at least $d_{\text{in}} - r$ dimensional subspace.

Matrix rank and the name Low-Rank Adaptation

LoRA replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

Why

Matrix rank and the name Low-Rank Adaptation

LoRA replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

Why do we call it **low-rank** adaptation?

Matrix rank and the name Low-Rank Adaptation

LoRA replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

Why do we call it **low-rank** adaptation?

This is because small r means the difference matrix $\Delta\boldsymbol{W}$ is low-rank (in linear algebra sense).

Matrix rank and the name Low-Rank Adaptation

LoRA replaces $\boldsymbol{W}_{\text{pretrain}}$ by $\boldsymbol{W}_{\text{pretrain}} + \Delta\boldsymbol{W}$, where $\Delta\boldsymbol{W} := \frac{\eta}{2}\boldsymbol{B}\boldsymbol{A}$.

Why do we call it **low-rank** adaptation?

This is because small r means the difference matrix $\Delta\boldsymbol{W}$ is low-rank (in linear algebra sense).

Specifically, $\mathrm{rank}(\Delta\boldsymbol{W}) \leq r$ in LoRA, if $\eta > 0$.

What we expect on LoRA

LoRA can only change an input in $S := \mathrm{span}(\text{rows of } \boldsymbol{A})$.

What we expect on LoRA

LoRA can only change an input in $S := \mathrm{span}(\text{rows of } \boldsymbol{A})$.

Hence, we hope that, after LoRA training, the subspace $S := \mathrm{span}(\text{rows of } \boldsymbol{A})$ covers the important input subspace in application.

What we expect on LoRA

LoRA can only change an input in $S := \mathrm{span}(\text{rows of } \boldsymbol{A})$.

Hence, we hope that, after LoRA training, the subspace $S := \mathrm{span}(\text{rows of } \boldsymbol{A})$ covers the important input subspace in application.

In practice, the idea works well. Simple LoRA training (backprop + SGD) can find such an important subspace and change the NN behavior as we want.

Note on implementation: r, α, and η

In the `peft` library, the LoRA is implemented in the following form:

$$\Delta \boldsymbol{W} := \eta \frac{\alpha}{r} \boldsymbol{B}\boldsymbol{A},$$

where $\alpha \in \mathbb{R}$ is a hyperparameter.

Note on implementation: r, α, and η

In the `peft` library, the LoRA is implemented in the following form:

$$\Delta \boldsymbol{W} := \eta \frac{\alpha}{r} \boldsymbol{B}\boldsymbol{A},$$

where $\alpha \in \mathbb{R}$ is a hyperparameter.

Multiplying $\frac{\alpha}{r}$ is mathematically redundant. In fact, in many LoRA cases, $\alpha = \frac{1}{2}r$ is fixed.

Note on implementation: r, α, and η

In the `peft` library, the LoRA is implemented in the following form:

$$\Delta \boldsymbol{W} := \eta \frac{\alpha}{r} \boldsymbol{B}\boldsymbol{A},$$

where $\alpha \in \mathbb{R}$ is a hyperparameter.

Multiplying $\frac{\alpha}{r}$ is mathematically redundant. In fact, in many LoRA cases, $\alpha = \frac{1}{2} r$ is fixed.

Hence, we use the following form:

$$\Delta \boldsymbol{W} := \frac{\eta}{2} \boldsymbol{B}\boldsymbol{A},$$

LoRA — Answer

Question: How can we modify the AI behavior effectively with a small number of parameters?

Answer:

LoRA — Answer

Question: How can we modify the AI behavior effectively with a small number of parameters?

Answer: We only change a matrix parameter behavior on the “important” low-dimensional linear subspace.

While

LoRA — Answer

Question: How can we modify the AI behavior effectively with a small number of parameters?

Answer: We only change a matrix parameter behavior on the “important” low-dimensional linear subspace.

While we give up a full rank update, it works well practically.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

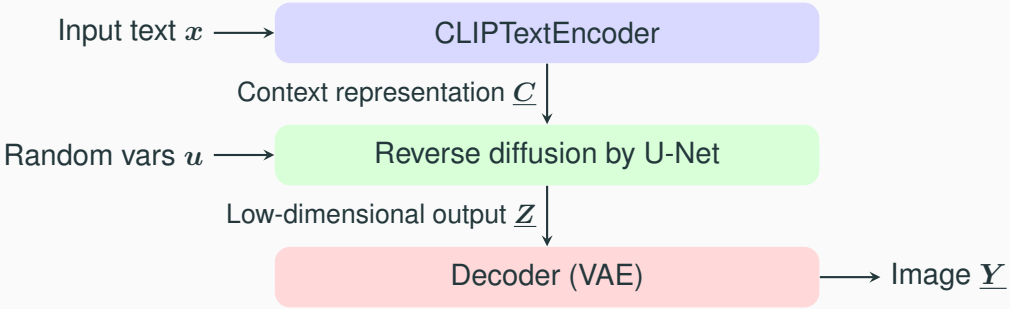
5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Question: LoRA can apply to matrix parameters in GenAI. Which parts

Question: LoRA can apply to matrix parameters in GenAI. Which parts are the targets of LoRA in a practical GenAI specifically?

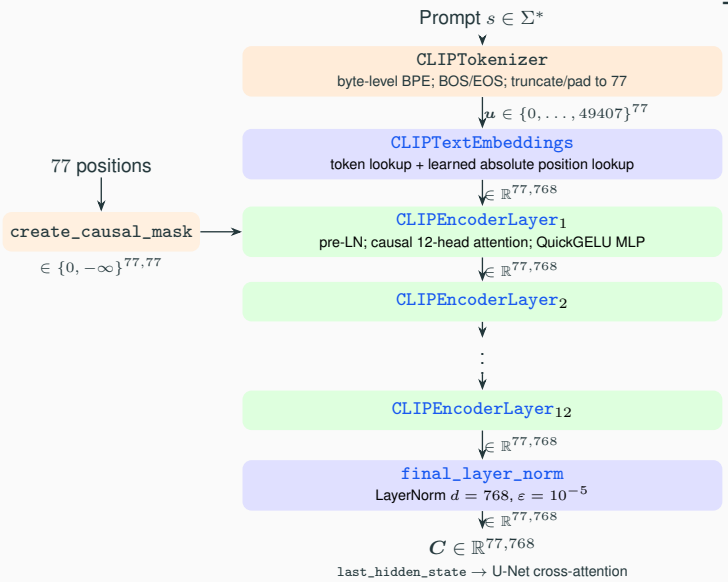
Review of Stable diffusion 1.5



In real PEFT situations, LoRA is applied to

- CLIP text encoder
- U-Net in the reverse diffusion

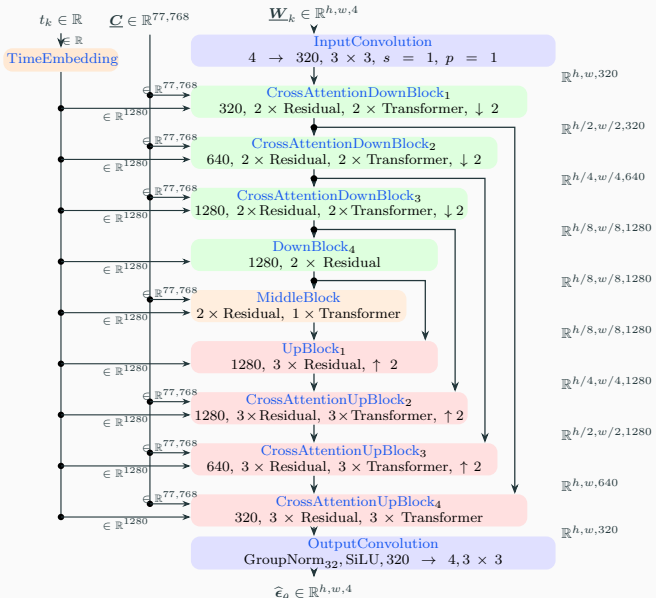
Stable Diffusion 1.5 text encoder: CLIP ViT-L/14 — Whole structure



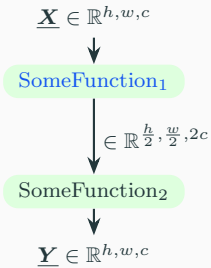
The structure is simple:

- Truncate/padding to 77 tokens
- Embedding
- Attention layers

Stable Diffusion 1.5 U-Net



Legend



Rounded boxes: functions.
Arrows: tensors; the adjacent superscripts give their shapes.
Function in ThisColor indicates it has learnable parameters.

$h = H/8, w = W/8$; image dimensions are divisible by 64.

U-Net outputs predicted noise.

Source: SD 1.5 U-Net configuration; Diffusers UNet2DConditionModel.

Which SD 1.5 layers contain this LoRA?

Take `anime_minimalist_v1-000020.safetensors` as an example.

Network	Target	Modules	$\boldsymbol{A}, \boldsymbol{B}$ entries
CLIP	Self-attention $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$	48	9,437,184
CLIP	MLP `fc1,fc2`	24	11,796,480
U-Net	Self-attention $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$	64	12,779,520
U-Net	Cross-attention $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$	64	12,730,368
U-Net	FFN `net.0.proj,net.2`	32	22,364,160
U-Net	`proj_in,proj_out` (1×1 conv)	32	6,389,760
	Total	264	75,497,472

They are all for token-wise linear operations.

It does **NOT** contain LoRA for the VAE or for U-Net ResNet 3×3 convolutions.

One U-Net Transformer: 12 LoRA modules

An SD 1.5 U-Net contains 16 spatial Transformer blocks: $2 + 2 + 2$ in down blocks, 1 in the middle, $3 + 3 + 3$ in up blocks.

- `proj_in` and `proj_out`: 2 LoRA modules;
- `attn1` (self): $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$, 4;
- `attn2` (cross): $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$, 4;
- `ff.net`: `0.proj` and 2, 2.

Hence $16 \cdot 12 = 192$ U-Net LoRA modules. The remaining $12 \cdot 6 = 72$ modules modify CLIP.

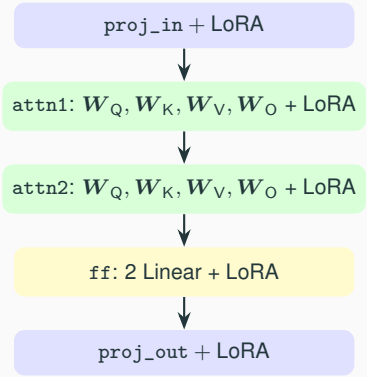


Diagram identifies targets; normalizations and residuals omitted.

Source: file tensor-key audit; https://github.com/CompVis/stable-diffusion/blob/main/ldm/modules/attention.py#L975-L1028;

CLIP: LoRA acts on text-token vectors

Let $t \in \{0, \dots, 76\}$ be a text-token position and $\boldsymbol{x}_t \in \mathbb{R}^{768}$ the CLIP hidden state at one of 12 encoder layers. CLIP has $H_{\text{clip}} = 12$ heads, so $d_h = 768/12 = 64$.

For $P \in \{\mathrm{Q}, \mathrm{K}, \mathrm{V}\}$, define independently

$$\boldsymbol{W}_P \in \mathbb{R}^{768,768},$$
$$\boldsymbol{A}_P \in \mathbb{R}^{128,768}, \quad \boldsymbol{B}_P \in \mathbb{R}^{768,128},$$
$$\boldsymbol{p}_t = (\boldsymbol{W}_P + \tfrac{1}{2}\boldsymbol{B}_P\boldsymbol{A}_P)\boldsymbol{x}_t \in \mathbb{R}^{768}.$$

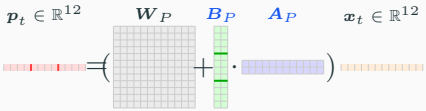


Illustration: 3 heads $\times$ 4 coordinates; rank 2.
$\boldsymbol{p}_t = (\boldsymbol{W}_P + \frac{1}{2}\boldsymbol{B}_P\boldsymbol{A}_P)\boldsymbol{x}_t$ (column vectors).

Split $\boldsymbol{p}_t$ into $\boldsymbol{p}_t^{[h]} \in \mathbb{R}^{64}$, $h \in \{1, \dots, 12\}$. This applies to $\boldsymbol{q}_t^{[h]}, \boldsymbol{k}_t^{[h]}, \boldsymbol{v}_t^{[h]}$.

The 12 heads do not have separately learned $\boldsymbol{A}_P^{[h]}$: $\boldsymbol{A}_P$ is shared among heads **within the same projection**.

Sources: text encoder config,

CLIP: multi-head attention and its other LoRA targets

For $h \in \{1, \dots, 12\}$ and token positions t, s, the causal attention score and probabilities are

$$a_{t,s}^{[h]} := \frac{(\boldsymbol{q}_t^{[h]})^\top \boldsymbol{k}_s^{[h]}}{\sqrt{64}}, \qquad p_{t,s}^{[h]} := \frac{e^{a_{t,s}^{[h]}}}{\sum_{u=0}^{t} e^{a_{t,u}^{[h]}}} \quad (0 \le s \le t).$$

$$\boldsymbol{o}_t^{[h]} := \sum_{s=0}^{t} p_{t,s}^{[h]} \boldsymbol{v}_s^{[h]} \in \mathbb{R}^{64}, \quad \boldsymbol{o}_t := \begin{bmatrix} \boldsymbol{o}_t^{[1]} \\ \vdots \\ \boldsymbol{o}_t^{[12]} \end{bmatrix} \in \mathbb{R}^{768}.$$

The output projection also has LoRA: $\boldsymbol{y}_t = (\boldsymbol{W}_\mathsf{O} + \frac{1}{2}\boldsymbol{B}_\mathsf{O}\boldsymbol{A}_\mathsf{O})\boldsymbol{o}_t$, where $\boldsymbol{W}_\mathsf{O} \in \mathbb{R}^{768,768}$, $\boldsymbol{A}_\mathsf{O} \in \mathbb{R}^{128,768}$, $\boldsymbol{B}_\mathsf{O} \in \mathbb{R}^{768,128}$.

Also, LoRA usually modifies both MLP linear maps: $\mathbb{R}^{768} \to \mathbb{R}^{3072} \to \mathbb{R}^{768}$.

Sources: https://github.com/huggingface/transformers/blob/main/src/transformers/models/clip/modeling_clip.py#L2555-L2588; https://github.com/huggingface/transformers/blob/main/src/transformers/models/clip/modeling_clip.py#L2590-L2675.

LoRA in U-Net

LoRA is applied to the following places in U-Net:

LoRA in U-Net

LoRA is applied to the following places in U-Net:

- In self-attention and cross-attention: applied to $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$, as we saw in CLIP.

LoRA in U-Net

LoRA is applied to the following places in U-Net:

- In self-attention and cross-attention: applied to $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$, as we saw in CLIP.
- Token-wise fully connected layer

LoRA in U-Net

LoRA is applied to the following places in U-Net:

- In self-attention and cross-attention: applied to $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$, as we saw in CLIP.
- Token-wise fully connected layer
- 1x1 convolution: it is also (essentially) token-wise fully connected layer.

LoRA in practice — Answer

Question: Which parts are the targets of LoRA in a practical GenAI specifically?

Answer:

LoRA in practice — Answer

Question: Which parts are the targets of LoRA in a practical GenAI specifically?

Answer: In Stable diffusion 1.5, LoRA is applied to token-wise matrix multiplications in the CLIP encoder and U-Net.

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Outline

1. Introduction

2. Preliminaries: Mathematical Notations

3. Why parameter-efficient parameter tuning?

4. How do we efficiently tune the parameter? LoRA

5. How do we apply PEFT (LoRA) in practical GenAI?

6. Summary

Summary

Summary

- PEFT can represent an AI behavior change with a few parameters. It saves storage cost and makes training easier.

Summary

- PEFT can represent an AI behavior change with a few parameters. It saves storage cost and makes training easier.
- Rank-r LoRA represents the fine-tuning by the product of an r column matrix and an r-row matrix. Small r saves the storage cost, while sacrificing the representability.

Summary

- PEFT can represent an AI behavior change with a few parameters. It saves storage cost and makes training easier.
- Rank-r LoRA represents the fine-tuning by the product of an r column matrix and an r-row matrix. Small r saves the storage cost, while sacrificing the representability.
- The targets of LoRA in SD 1.5 are token-wise linear operations in CLIP and U-Net, e.g., $\boldsymbol{W}_{\mathrm{Q}}, \boldsymbol{W}_{\mathrm{K}}, \boldsymbol{W}_{\mathrm{V}}, \boldsymbol{W}_{\mathrm{O}}$ in attention layers, token-wise MLP/FFN, 1x1 convolutional layers. LoRA does not apply to VAE or 3x3 convolutional layers.

References i

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville.
Deep Learning.
MIT Press, 2016.

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen.
LoRA: Low-rank adaptation of large language models.
In International Conference on Learning Representations, 2022.

References ii

[3] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel.
Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning.
In Advances in Neural Information Processing Systems, volume 35, 2022.